

Operating Systems

Dr. Mostafa Elgendy

Computer Science Department,

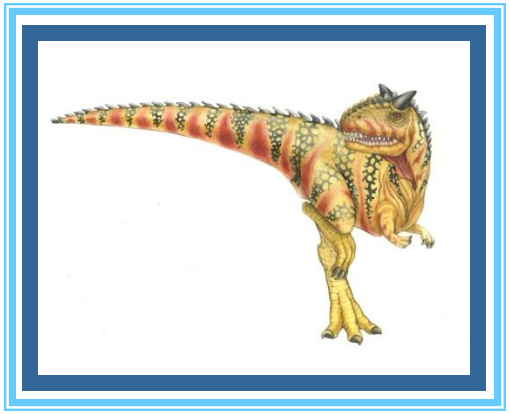
Faculty of Computers and Artificial Intelligence

Benha University

2022



Chapter 3
Processes- Part 1



Silberschatz, Galvin and Gagne ©2018



Outline

- Process Concept.
- Process Scheduling.
- Operations on Processes.





Process Concept





Process Concept

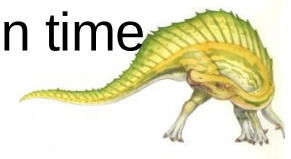
- **Program** is passive entity stored on disk (executable file); process is active
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program





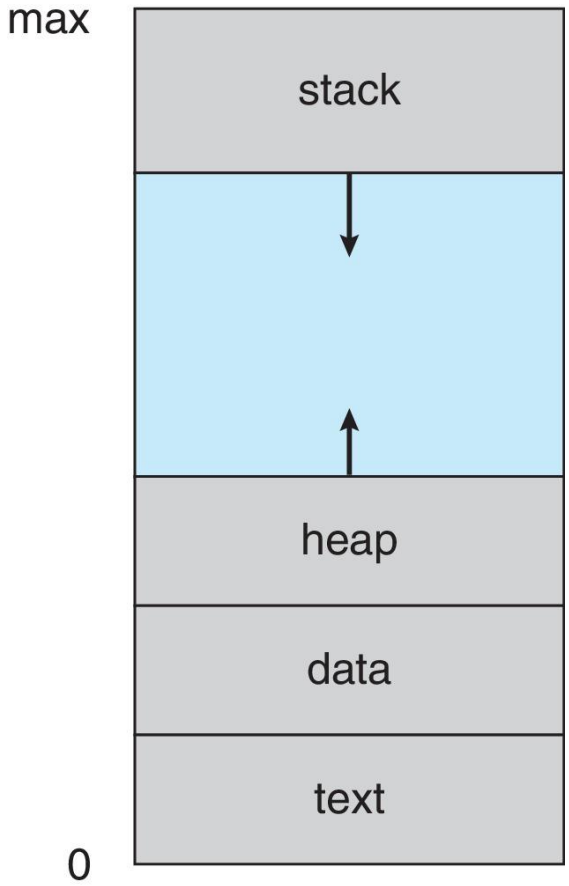
Process Concept (Cont.)

- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity: **program counter**, **processor registers**
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time





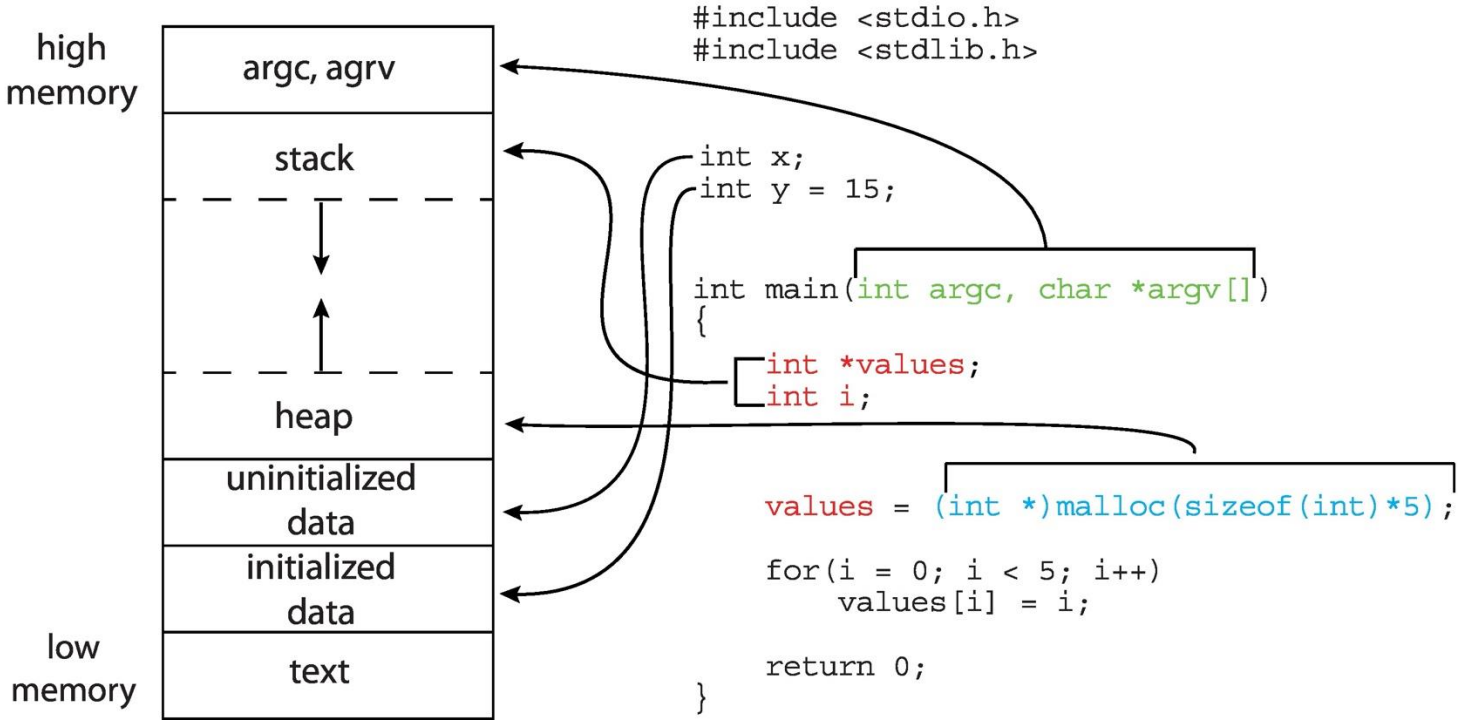
Process in Memory







Memory Layout of a C Program







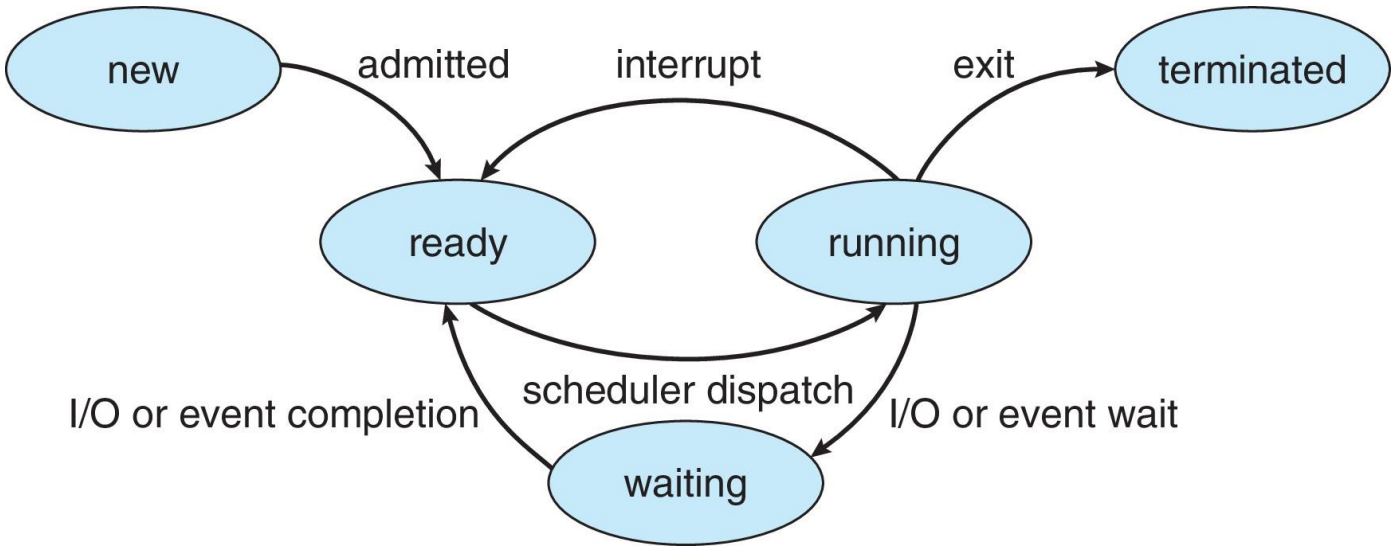
Process State

- As a process executes, it changes state
  - **New:** The process is being created
  - **Running:** Instructions are being executed
  - **Waiting:** The process is waiting for some event to occur
  - **Ready:** The process is waiting to be assigned to a processor
  - **Terminated:** The process has finished execution





Diagram of Process State

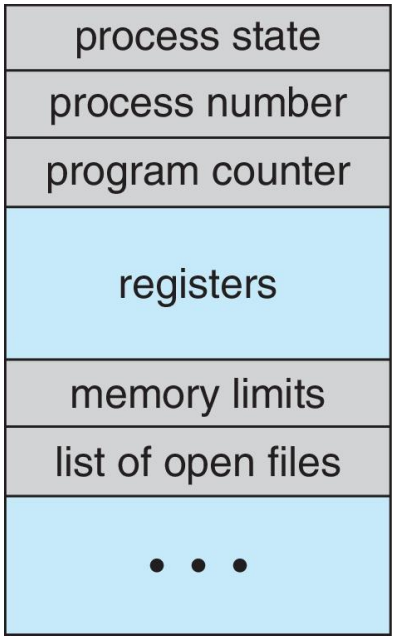


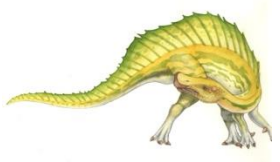




Process Control Block (PCB)

- Information associated with each process(also called task control block):
  - **Process state** – running, waiting, etc.
  - **Program counter** – location of instruction to next execute
  - **CPU registers** – contents of all process-centric registers
  - **CPU scheduling information-** priorities, scheduling queue pointers
  - **Memory-management information** – memory allocated to the process
  - **Accounting information** – CPU used, clock time elapsed since start, time limits
  - **I/O status information** – I/O devices allocated to process, list of open files



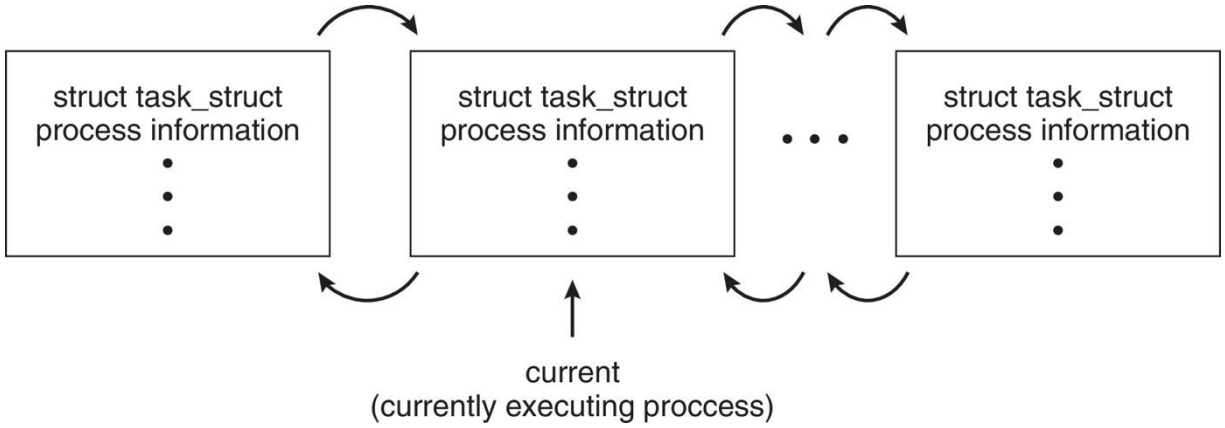




Process Representation in Linux

- Represented by the C structure **task_struct**

```
pid t_pid;                          /* process identifier */
long state;                         /* state of the process */
unsigned int time_slice   /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;     /* address space of this process */
```







Process Scheduling





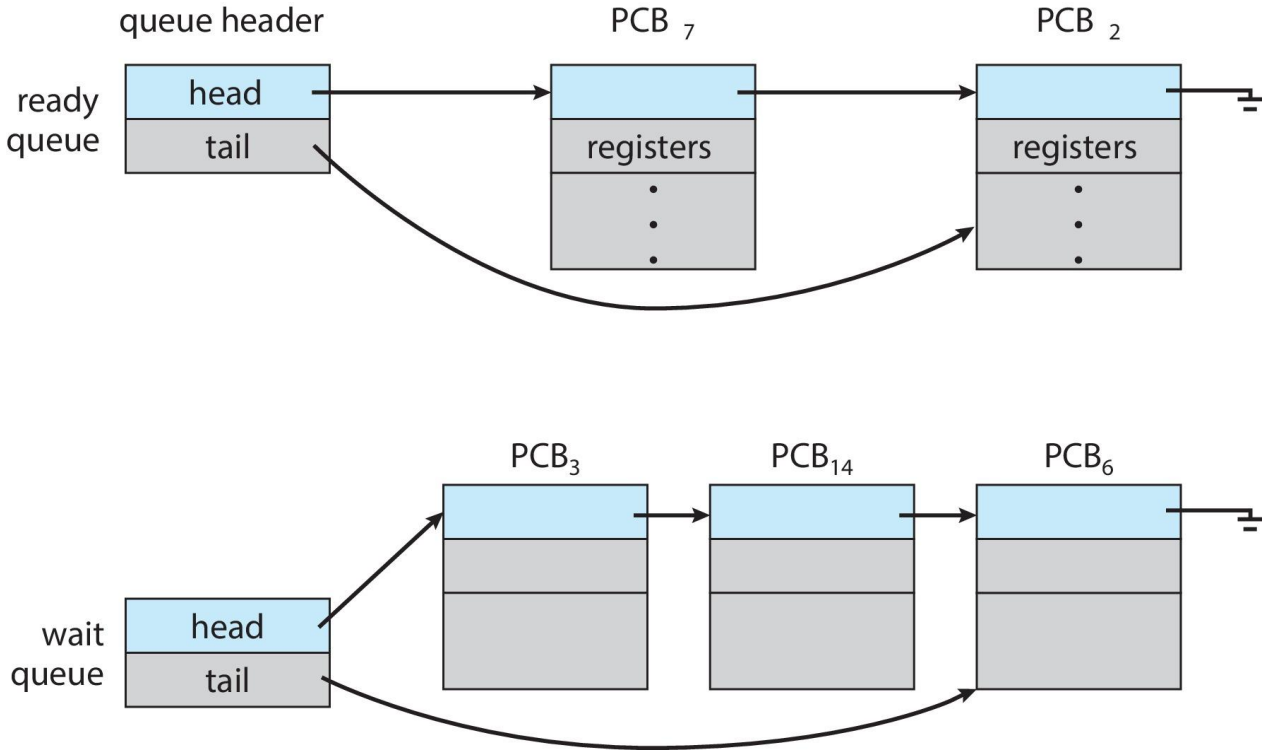
Process Scheduling

- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Wait queues** – set of processes waiting for an event (i.e., I/O)
  - Processes migrate among the various queues





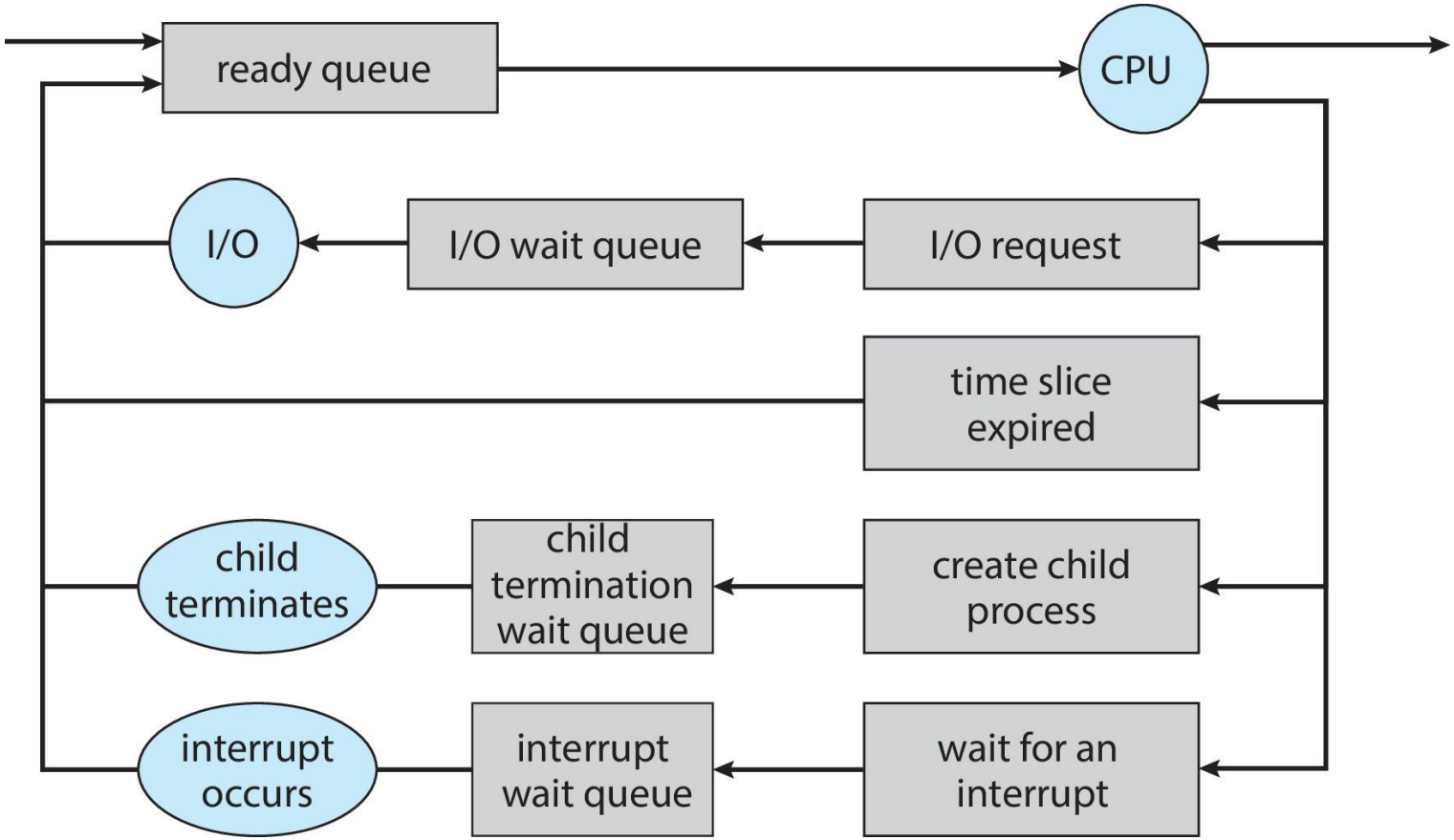
Ready and Wait Queues







Representation of Process Scheduling

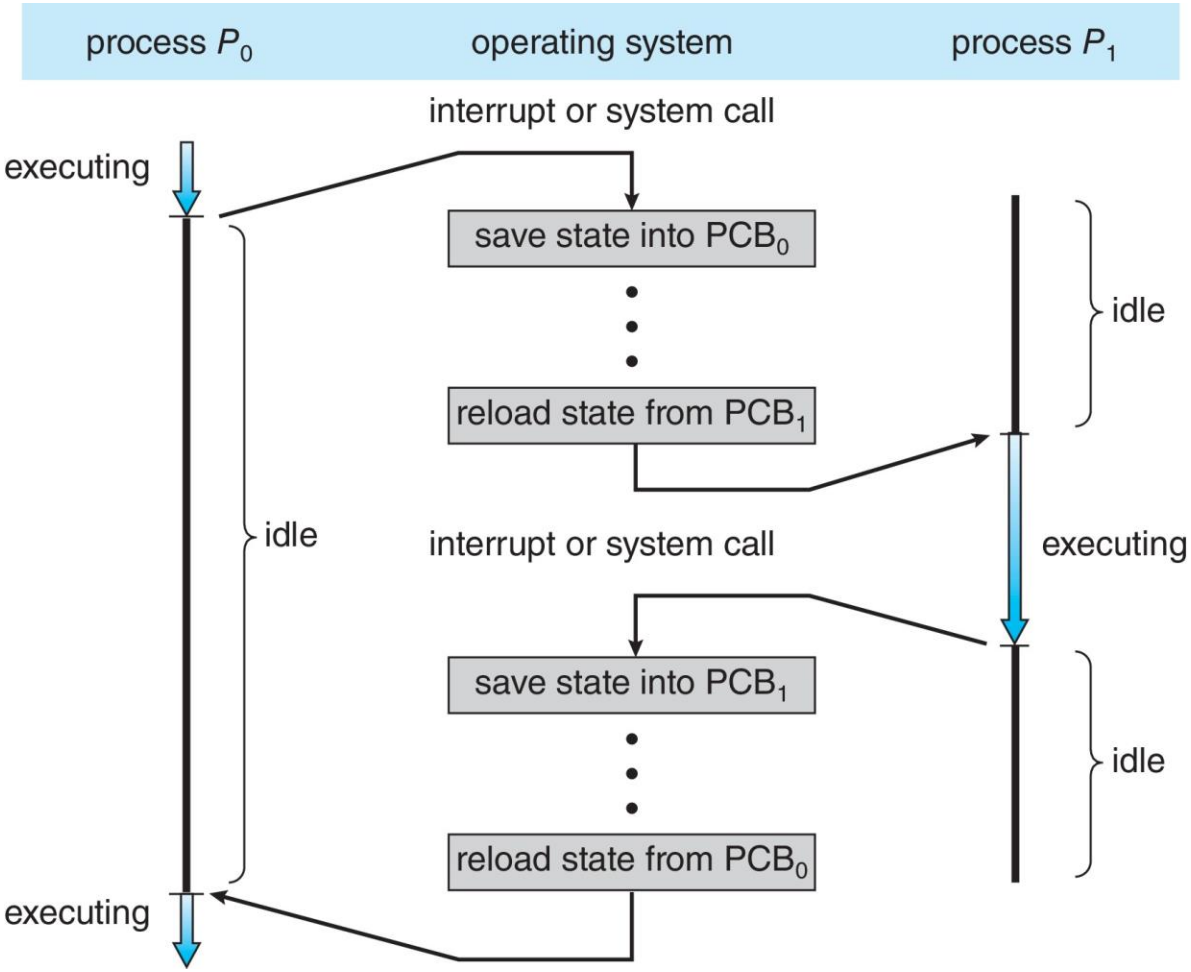






CPU Switch From Process to Process

- A context switch occurs when the CPU switches from one process to another.







Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load **the saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once





Operations on Processes





Operations on Processes

- System must provide mechanisms for:
  - Process creation
  - Process termination





Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate





Process Creation (Cont.)

- UNIX examples
  - **fork()** system call creates new process
  - **exec()** system call used after a **fork()** to replace the process' memory space with a new program
  - Parent process calls **wait()** waiting for the child to terminate





Process Termination

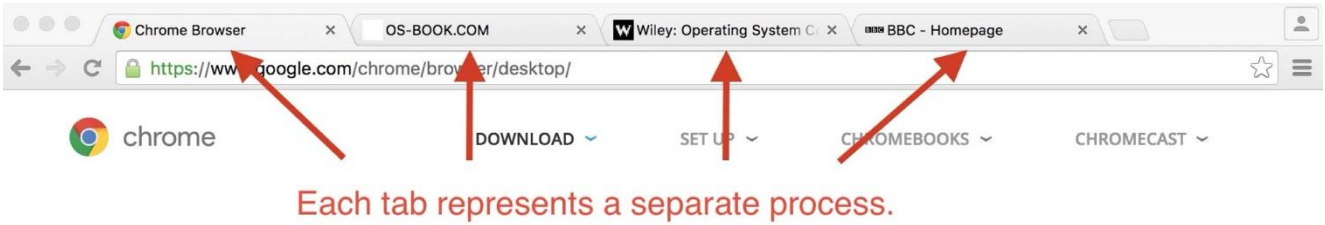
- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
  - Returns status data from child to parent (via **wait())**
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call.
- Some reasons for doing so:
  - Child has exceeded allocated resources.
  - Task assigned to child is no longer required.
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates.





Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript.
    - A new renderer created for each website opened
  - **Plug-in** process for each type of plug-in







THANK YOU

